

Usage of void *pvParameters:

void *pvParameters pointer is one of the most powerful features of xTaskCreate().

To understand why it's required, think about standard C functions: if you want to pass data into a function, you just write my_function(int x, float y).

But with an RTOS, you never call the task function yourself. The FreeRTOS scheduler calls it behind the scenes when the task boots up. Because the scheduler has to be able to launch *any* custom function you write, the creators of FreeRTOS needed a universal, generic way to pass data into a task.

In C, a void * (void pointer) is a generic pointer that can point to absolutely anything—an integer, a string, a structure, or an entire data array.

Here are the two main reasons why it is required:

Reason 1: Reusability (Writing One Function for Multiple Hardware Instances)

Reason 2: Strict Function Signature (The Contract)

Code:

```
#include<stdio.h>
#include "freertos/FreeRTOS.h"
#include "freertos/task.h"

typedef struct{
    char *task_message;
    int delay_ms;
} Task_Config_t;

void generic_print_task(void *pvParameters){
    Task_Config_t *myconfig = (Task_Config_t *) pvParameters;
    while(1){
        printf("%s \n", myconfig-> task_message);
        vTaskDelay(myconfig->delay_ms / portTICK_PERIOD_MS);
    }
}

Task_Config_t task1_data = { .task_message = "[Instance 1] Fast task",
    .delay_ms = 400};
Task_Config_t task2_data = { .task_message = "[Instance 2] Slow task",
    .delay_ms = 1500};

void app_main(void){
    xTaskCreate(
        generic_print_task,
        "Fast print",
```

```

        2048,
        &task1_data,
        5,
        NULL
    );

    xTaskCreate(
        generic_print_task,
        "Slow print",
        2048,
        &task2_data,
        5,
        NULL
    );
}

```

Output:

```

I (228) heap_init: At 4008A3E0 len 00015C20 (87 KiB): IRAM
I (234) spi_flash: detected chip: generic
I (237) spi_flash: flash io: dio
W (240) spi_flash: Detected size(4096k) larger than the size in the binary image header(2048k). Using the
I (253) main_task: Started on CPU0
I (253) main_task: Calling app_main()
[Instance 1] Fast task
[Instance 2] Slow task
I (253) main_task: Returned from app_main()
[Instance 1] Fast task
[Instance 1] Fast task
[Instance 1] Fast task
[Instance 2] Slow task
[Instance 1] Fast task
[Instance 1] Fast task
[Instance 1] Fast task
[Instance 1] Fast task
[Instance 2] Slow task
[Instance 1] Fast task
[Instance 1] Fast task
[Instance 1] Fast task
[Instance 1] Fast task
[Instance 2] Slow task
[Instance 1] Fast task
[Instance 1] Fast task
[Instance 1] Fast task
[Instance 1] Fast task
[Instance 2] Slow task
[Instance 1] Fast task
[Instance 1] Fast task
[Instance 1] Fast task
[Instance 2] Slow task
[Instance 1] Fast task
[Instance 1] Fast task

```

The FreeRTOS scheduler allocates completely separate chunks of Stack memory (2048 bytes each) for these two instances, meaning they execute completely independently even though they share the exact same underlying logic lines.